Quatrième partie

Langages et Programmation

Chapitre (2).1

[Python]- Création de Modules et de Packages

Remarque préliminaire : Sauf mention contraire, la version de Python utilisée sera toujours postérieure à Python 3.10. Les programmes seront exécutés dans un environnement de type Unix (macOS ou Linux).

Sources :

Sites web :	https://docs.python.org/3.12/tutorial/modules.html
Sites web :	https://github.com/pypa/pip
Livre :	*Python 3 - Les fondamentaux du langage*, CHAZALET, ENI Editions
Livre électronique :	*Une introduction à Python 3*, CORDEAU & POINTAL, Creative Commons BY-NC-SA-3
Livre électronique :	*A Python Book*, KUHLMAN, Open Source MIT License

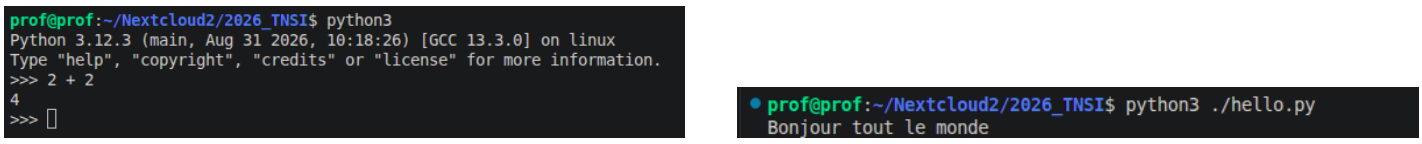
1 Mon environnement de travail

Pour écrire et exécuter du code Python, il faut a minima deux programmes : *un éditeur de texte* et *un interpréteur* du langage Python. Parfois, ces deux programmes sont réunis dans une seule application et ils constituent **votre IDE** (*Integrated Development Environment*) : c'est votre environnement de travail !

À savoir

Le choix d'un IDE dépend des préférences de l'utilisateur, mais aussi parfois de la plateforme sur laquelle il travaille. Il est donc conseillé de privilégier les IDE "multiplateformes" c'est-à-dire compatibles avec Windows, Linux, Mac, FreeBSD, etc. Ce cours a été rédigé avec l'IDE VSCodium.

Un shell (une console, un interprète) est directement accessible via la commande *python* (ou *python3*) depuis *une console* ; on peut quitter le shell python avec la commande *quit()* (fig. 1.2.a). Il est même possible d'exécuter du code en passant le chemin vers le fichier comme argument (fig. 1.2.b) :



(a) (b)

FIGURE 1 – (a) Commande Python & (b) python <nom ou chemin du fichier>

Certains éditeurs de texte sont particulièrement aboutis et rendent l'écriture du code plus agréable en ajoutant des couleurs, des numéros de lignes, l'autocomplétion, ... Certains sont des IDE à part entière puisqu'ils permettent d'écrire et d'exécuter le code (VSCodium, Visual Studio Code, ...). À vous de choisir celui qui vous convient ! Voici le résultat d'un sondage présentant les éditeurs de code les plus populaires en 2025 : on y retrouve par exemple **Visual Studio Code, Notepad++, Vim, Sublime Text, PyCharm, etc.**
https://survey.stackoverflow.co/2025/technology#most-popular-technologies-dev-envs-dev-envs

Si vous recherchez un IDE, alors vous pourrez tester **Thonny** qui est l'IDE choisi pour les épreuves pratiques.

À savoir

Les *notebooks* de **Jupyter** sont très adaptés à la réalisation de projets et à la prise de notes : c'est cet IDE qui sera privilégié en classe de terminale pour les séances de cours.

2 Qu'est-ce qu'un module ?

2.1 Activité introductive : pourquoi séparer un programme en plusieurs fichiers ?

Jusqu'à présent, vous avez principalement écrit vos programmes dans un seul fichier. Pourtant, lorsque le code devient plus long ou que certaines fonctions doivent être réutilisées, il peut être utile de répartir le code dans plusieurs fichiers.

Considérons les deux fichiers suivants, placés dans un même dossier.

Fichier `outil.py`

```
1 def saluer(nom):
2     print("Bonjour", nom)
3
4 print("Le module outil est chargé")
```

Fichier `main.py`

```
1 import outil
2
3 print("Début du programme principal")
4 outil.saluer("Alice")
```

✔ **Questions**

1. Sans exécuter le programme `main.py`, prévoyez l'ordre dans lequel les trois messages seront affichés.
2. Exécutez ensuite `main.py` depuis un terminal. Comparez le résultat avec votre prévision.
3. Pourquoi le message « Le module outil est chargé » apparaît-il alors qu'il n'y a pas d'appel à `print` dans `main.py` à cet endroit ?
4. Quel est l'intérêt de placer la fonction `saluer` dans un fichier séparé ?

À retenir : lorsqu'un module est importé pour la première fois, Python exécute les instructions qui se trouvent dans ce module. Nous verrons comment éviter que certaines instructions de test s'exécutent lors d'un import.

2.2 Notion de programme principal

Dans cette partie, nous allons présenter les notions de *module*, de *package*, de *librairie* et de *framework*. Bien que certains des termes précédents soient parfois considérés comme équivalents, nous adopterons les définitions suivantes :

Définition n°1

Un module Python est un ensemble de code regroupé dans un même espace de noms. Dans les situations que nous étudions, il correspond généralement à un fichier portant l'extension .py. Il peut contenir des instructions, des variables, des fonctions et des classes.

Remarque : L'intérêt d'un module est d'être à la fois **directement exécutable** et aussi "**importable**" c'est-à-dire réutilisable dans un autre fichier. Pour rendre un module aussi réutilisable que possible, il est important de définir des *espaces de noms* comme les fonctions ou les classes. L'utilisation d'un module se fait à l'aide du mot clé **import** (on dit parfois *la primitive* **import**).

Commençons par créer notre premier module que nous nommerons **je_suis_un_module.py**. Le code Python est présenté ci-après :

Exemple n°1

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Created on Sun Jul 12 10:43:18 2020
5  @author: MoonsieurNSI
6  """
7  def premierModule():
8      print("Voici mon premier module")
9
10 premierModule()
```

Commentaires

Lignes 1 et 2 : on indique le *shebang* (la commande à utiliser) et l'encodage du fichier.
Lignes 3 et 4 : on écrit une *docstring*.
Lignes 5 et 6 : on définit notre unique fonction.
Ligne 8 : on appelle la fonction **premierModule**.

Lors de la création du module, deux variables globales sont associées au fichier : `__doc__` et `__name__`. Le premier contient la docstring principale (généralement en tête de module) tandis que le second contient une chaîne qui dépend **du contexte** dans lequel le script est appelé.

Vocabulaire

Cette année, nous rencontrerons de nombreux noms d'objets Python : fonctions, méthodes, attributs, objets, classes, etc. dont le nom commence (et parfois se termine) par **un double underscore**.
En fonction de votre éditeur de texte, le **double underscore** peut apparaître plus ou moins compact :

___maVariable___ ou __maVariable__

Selon le contexte et les habitudes, ces variables sont parfois surnommées *variables magiques* ou *dunders* : nous choisirons de les appeler **variables spéciales** comme souvent dans la littérature française sur le langage Python.

Affichons le contenu de ces deux variables dans le cas d'**une exécution directe** via le terminal par exemple ; pour cela, on supposera que le fichier **je_suis_un_module.py** est enregistré dans le répertoire courant et que la ligne 6 de notre Exemple n°1 est remplacée par :

```
print(__doc__, '\n', __name__)
```

✔ **Question !**

— Quel est le rôle du caractère '\n' ?

Terminal Linux

```
the_brat@LinuxMint:~$ python je_suis_un_module.py
Created on Sun Jul 12 10:43:18 2020
@author: the_brat
 __main__
```

Comme prévu, la variable`__doc__` contient la docstring principale qui est affichée dans la console. Par défaut, la seconde variable `__name__` contient l'étrange **chaîne de caractères** `'__main__'`. Nous allons présenter son rôle dans quelques lignes.

À savoir

Le mot clé `import` est utilisé pour importer un module dans un fichier Python. **Le nom du module est le nom du fichier sans l'extension ".py"** ; par exemple, si le fichier **je_suis_un_module.py** est importé comme module alors son nom est simplement **je_suis_un_module**.

Dans ce deuxième exemple, nous rédigeons du code Python dans un fichier principal nommé **mon_projet.py** dans lequel nous importons le module **je_suis_un_module** (légèrement modifié).

Exemple n°2 : mon_projet.py

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 import je_suis_un_module
4 print(f"__name__ => {__name__}")
```

Le module je_suis_un_module

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """Created on Sun Jul 12 10:43:18 2020
4 @author: the_brat"""
5 def premierModule():
6     print(f"__name__ => {__name__}")
7
8 premierModule()
```

Exécutons directement le programme principal "mon_projet.py" dans un terminal Linux et analysons le résultat observé :

Terminal Linux

```
the_brat@LinuxMint:~$ python ./mon_projet.py
__name__ => je_suis_un_module
__name__ => __main__
```

Le plus important ici est de remarquer que deux lignes sont renvoyées par le programme **mon_projet.py** ; ceci peut paraître étrange car ce *script* ne présente qu'un seul appel à la fonction **print** (à la ligne 4 du code).

À savoir

Quand un module est importé, celui-ci est exécuté entièrement.

Dès lors, que se passe-t-il avec le programme **mon_projet.py** ?

- ♠ Ligne 1-2 sont les commentaires (*shebang* et *encodage*)
- ♠ Ligne 3 : le module **je_suis_un_module** est importé et donc exécuté.
  - ♣ Ligne 8 du module, on appelle la fonction **premierModule**
  - ♣ Ligne 6, on appelle la fonction **print**. C'est ici que la fonction **print** est appelée pour la première fois !
  - ♣ Ligne 7, il n'y a pas de **return** ce qui signifie que la fonction renvoie **None** au **programme principal**.
- ♠ Ligne 4 : L'exécution du module est terminée et le programme principal appelle à son tour la fonction **print**.

Dans des langages de programmation tels que **C** ou **Java**, il existe une fonction nommée **main()** qui est exécutée lors du lancement du **programme principal**. En Python, il n'existe pas de fonction `main()` imposée par le langage : Python exécute les instructions du fichier lancé. C'est la variable spéciale `__name__` qui permet de différencier un script exécuté comme programme principal ou comme module importé.

À savoir

La variable spéciale `__name__` est du type `str` et contient toujours la valeur `'__main__'` **quand le script est exécuté comme programme principal** ; sinon, `__name__` contient le nom du module importé.

Quand on crée un fichier destiné à devenir un module importable, il est donc usuel de terminer le code par un test conditionnel portant sur le contenu de la variable **__name__**. Ceci permet d'éviter que du code soit automatiquement exécuté dans le programme principal.

Généralement, on écrit en fin de module le test suivant :

```
if __name__ == "__main__":
    #code à exécuter si le module est le programme principal
```

☞ **À votre tour !**

— **Ajouter un test au module je_suis_un_module** afin de ne pas exécuter la fonction **premierModule** quand ce programme est importé.
— Vérifier en exécutant d'abord **mon_projet.py** puis **je_suis_un_module.py**

2.3 Importer et utiliser un module

En Python (comme dans presque tous les langages), **le point sert d'accesseur**. C'est un terme compliqué pour signifier que si on souhaite accéder à un objet dans un conteneur alors on écrit :

conteneur.objet

Un exemple : supposons que dans le programme principal **mon_projet.py**, nous souhaitions utiliser la fonction **premierModule** contenue dans le module **je_suis_un_module**. Voici une syntaxe possible :

Exemple n°3 : mon_projet.py

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 import je_suis_un_module
4
5 je_suis_un_module.premierModule()
```

Le module je_suis_un_module

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """Created on Sun Jul 12 10:43:18 2020
4 @author: the_brat"""
5 def premierModule():
6     print("Vivement qu'on m'utilise !")
7 if __name__ == "__main__":
8     print("Ceci est un test !")
9     premierModule()
```

☞ **À votre tour !**

— Exécuter le programme principal **mon_projet.py**.

✔ **Question !**

— Pour quelle raison la ligne 8 du module n'est-elle pas exécutée ?

La notation peut paraître lourde, en particulier si le nom du module est peu pratique, comme c'est le cas ici. Python propose l'utilisation d'**un alias** avec l'utilisation du mot clé **as**. Une autre façon de procéder consiste à importer uniquement les objets qui nous intéressent à l'aide du mot clé **from** : dans ce cas on peut directement appeler l'objet par son nom sans utiliser d'accesseur. Les deux exemples ci-dessous illustrent ces principes :

Exemple n°4 : mon_projet.py

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 import je_suis_un_module as je
4
5 je.premierModule()
```

Exemple n°5 : mon_projet.py

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 from je_suis_un_module import premierModule
4
5 premierModule()
```

☞ **À votre tour !**

— Exécuter le programme principal **mon_projet.py** en utilisant les différentes méthodes présentées

Le module **je_suis_un_module** ne contient qu'une seule fonction. Toutefois lorsque cela est nécessaire, Python permet l'importation de plusieurs *objets* en séparant les noms par des virgules. Voici la syntaxe :

```
from je_suis_un_module import fonction1, fonction2, fonction3
```

Remarque : L'utilisation du mot clé `from` doit attirer l'attention. En effet, si on importe plusieurs modules de cette façon et qu'*un objet* portant le même nom existe dans plusieurs de ces modules, alors il risque d'y avoir un conflit. De plus, on rencontre parfois une syntaxe utilisant *la wild card* ***** permettant a priori d'importer tous les objets d'un module. Cette méthode est plutôt déconseillée pour les mêmes raisons car elle accroît ce risque en important tous *les objets*. Voici la syntaxe :

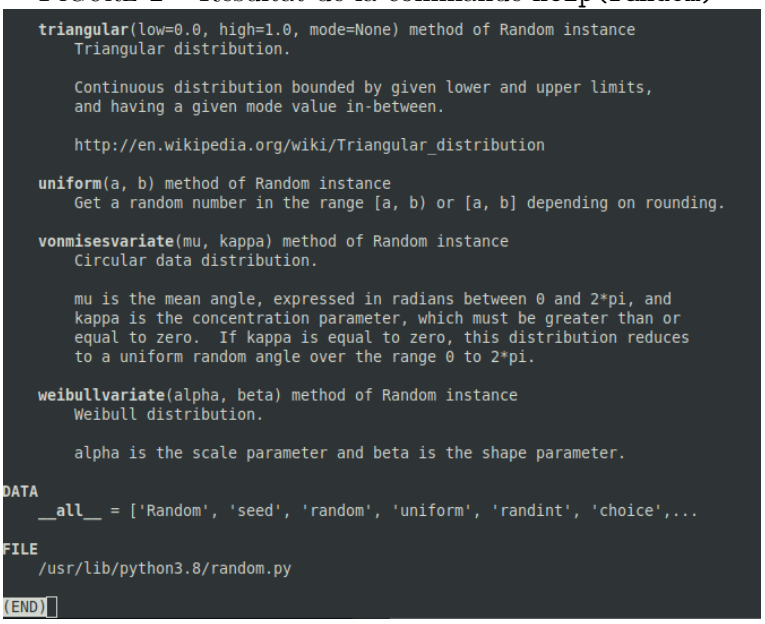
```
from je_suis_un_module import *
```

À savoir

Un module (ou les objets d'un module) peut être importé n'importe où dans le programme principal, y compris à l'intérieur d'une fonction par exemple. Cependant, par convention, on écrit généralement les lignes d'importation en début de programme en commençant par les modules de la **bibliothèque standard** (*library* en anglais) puis les modules tiers puis les modules personnels.

A ce stade, il est important d'ajouter une notion cruciale du programme NSI qui a été développée dans le chapitre sur *la programmation orientée objet* : c'est la notion d'**interface**. Ainsi, quand on souhaite importer un module, il faut connaître les éléments qu'il met à disposition. Une méthode simple consiste à rechercher le fichier ".py" relatif à notre module, puis de l'ouvrir avec un éditeur de texte afin d'en explorer le contenu. Ce n'est pas une mauvaise méthode, mais elle peut s'avérer très pénible à mettre en œuvre ; en général, les concepteurs d'un module fournissent une interface (**API** : *Application Programming Interface*) afin de simplifier son utilisation par un autre programmeur ou un autre programme. Dans notre exemple, c'est la docstring du module ainsi que les fonctions intégrées `help` et `dir` peuvent servir d'interface.

FIGURE 2 – Résultat de la commande `help(random)`



☞ À votre tour !

— Ouvrir une console Python, puis importer le module `random`.
— Lancer la commande `help(random)` et écrire un programme qui affiche un nombre entier pseudo-aléatoire entre 0 et 100.
— Appuyer sur la touche q pour quitter la console en mode "help"

Parmi les modules de la bibliothèque standard (fournis avec Python), on peut citer :

— `math` qui propose des outils mathématiques standards
— `os` qui assure le dialogue avec le système d'exploitation

— `random` qui fournit des outils pour générer des nombres pseudo-aléatoires
— `sys` qui permet notamment d'interagir avec l'interpréteur Python
— `time` qui est dédié à la gestion du temps
— `turtle` qui offre la possibilité de dessiner
— `urllib` qui permet de récupérer des données depuis Internet

Vous trouverez tous les modules de **la bibliothèque standard** sur le site officiel de python : https://docs.python.org/

2.4 Chemin vers les modules

Jusqu'à présent, nous avons soigneusement évité d'aborder la question de la localisation des modules par rapport au fichier contenant le programme principal. Le mot clé `import` peut sembler magique puisqu'il suffirait de donner un nom pour que Python sache où chercher le module. Mais comment Python sait-il où se trouve le module ?

Pour retrouver un module lors d'un `import`, Python consulte une liste de chemins de recherche accessible dans le programme grâce à `sys.path`. Cette liste contient notamment des emplacements liés au lancement de Python et ceux où sont installés les modules.

La variable d'environnement `PYTHONPATH` permet, si nécessaire, d'indiquer des dossiers supplémentaires dans lesquels Python doit rechercher des modules. Elle est lue au démarrage de l'interpréteur : ses chemins sont alors ajoutés à `sys.path`.

Console Python

```
>>> import sys
>>> print(sys.path)
['/usr/share/texlive/texmf-dist/scripts/pythontex', '/usr/lib/python312.zip',
↪  '/usr/lib/python3.12', '/usr/lib/python3.12/lib-dynload',
↪  '/usr/local/lib/python3.12/dist-packages', '/usr/lib/python3/dist-packages',
↪  '/usr/lib/python3.12/dist-packages', '/home/prof/Nextcloud2/2026_TNSI/tex']
```

Lorsque Python rencontre une instruction `import`, il recherche le module dans les emplacements indiqués par `sys.path`. Pour un module écrit en Python, il peut notamment rechercher un fichier nommé `nom_du_module.py` ou un package correspondant. Si aucun module importable portant ce nom n'est trouvé, Python déclenche une erreur `ModuleNotFoundError`.

À savoir

Selon la manière dont Python est lancé, le dossier du script ou le dossier courant figure parmi les emplacements de recherche. Pour nos premiers exemples, nous placerons le programme principal et le module dans le même dossier afin que Python trouve le module.

Remarque : Il arrive souvent que l'on ait besoin d'installer un module ou un package tiers. Bien qu'elle ne soit pas installée avec Python, l'outil **pip** est couramment utilisé pour installer des packages Python. Il suffit d'ouvrir un terminal et de lancer la commande suivante :

pip install <nom du module>

Sur votre machine, le package sera généralement installé dans le dossier *sites-packages* ou *dist-packages* ; ils font partie des emplacements où Python recherche les modules. Vous trouverez plus d'informations sur le site officiel de l'application : https://pip.pypa.io/en/stable/installing/

☞ **À votre tour !**

— Ouvrir un terminal, puis installer le package **numpy**.
— Consulter l'aide relative à ce package

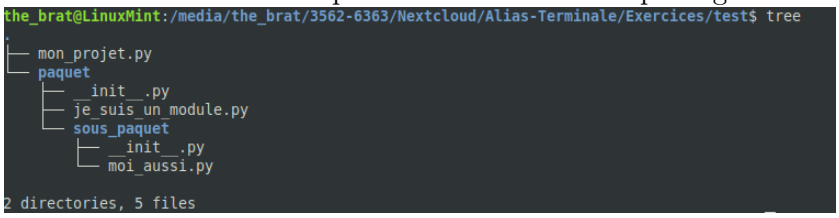
2.5 Packages, bibliothèques et frameworks

Définition n°2

Un package (ou *paquet*) est un dossier contenant un fichier éventuellement vide nommé `__init__.py` et des modules. Ce dossier peut lui-même contenir des "sous-paquets".

Considérons l'arborescence suivante, obtenue à l'aide de la commande Bash `tree` dans un terminal Linux :

FIGURE 3 – Exemple d'arborescence d'un package



✔ Question !

— Combien ce dossier contient-il de sous-packages et de modules ?

Considérons les modules suivants correspondant à l'arborescence précédente :

Le module je_suis_un_module

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
def premierModule():
    print("Vivement qu'on m'utilise !")

if __name__ == "__main__":
    premierModule()
```

Le module moi_aussi

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
def secondModule():
    print("Bonjour tout le monde !")

if __name__ == "__main__":
    secondModule()
```

Le programme principal reste **mon_projet.py** avec les modifications présentées ci-dessous ; notez en particulier le changement sur les **import** : on utilise l'accesseur (le point) jusqu'aux fonctions que l'on souhaite importer.

Exemple n°6 : mon_projet.py

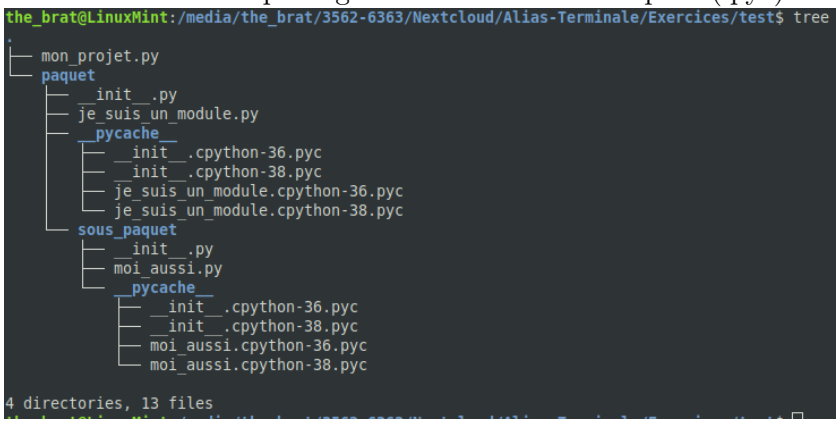
```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import sys
from paquet.je_suis_un_module import premierModule
from paquet.sous_paquet.moi_aussi import secondModule
print(sys.path)
premierModule()
secondModule()
```

☞ À votre tour !

— Ouvrir un terminal et construire l'arborescence de la figure 1.5
— Exécuter le programme **mon_projet.py** depuis le terminal

Remarque : Après une première exécution, Python peut avoir modifié l'arborescence : il met en cache le bytecode de certains modules importés (**.pyc**) et les conserve dans un dossier **__pycache__** afin d'accélérer les importations suivantes.

FIGURE 4 – Un package avec les modules compilés (.pyc)



Définition n°2

Une bibliothèque (ou *library*) est le nom généralement donné à un ensemble de packages, tandis qu'un **framework** est un ensemble de bibliothèques. Dans ce cours, on retiendra donc les distinctions suivantes :

module, package, bibliothèque et framework : quatre notions distinctes

Annexe A

Liens utiles

FIGURE 1 – 2026 — Éditeurs et IDE populaires

	Visual Studio Code	https://code.visualstudio.com/Download	Windows / macOS / Linux
	Notepad++	https://notepad-plus-plus.org/downloads/	Windows
	Vim	https://www.vim.org/download.php	macOS / Linux
	Sublime Text	https://www.sublimetext.com/3	Windows / macOS / Linux
	PyCharm	https://www.jetbrains.com/pycharm/download/	Windows / macOS / Linux
	Jupyter	https://jupyter.org/install	Windows / macOS / Linux
	Thonny	https://thonny.org/	Windows / macOS / Linux